ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА» (СПбГУТ)

АРХАНГЕЛЬСКИЙ КОЛЛЕДЖ ТЕЛЕКОММУНИКАЦИЙ ИМ. Б.Л. РОЗИНГА (ФИЛИАЛ) СПбГУТ (АКТ (ф) СПбГУТ)

КУРСОВОЙ ПРОЕКТ

НА ТЕМУ

РАЗРАБОТКА ПОДСИСТЕМЫ УЧЕТА НОМЕРНОГО ФОНДА ОБЩЕЖИТИЯ

Л109. 25КП01. 006 ПЗ

(Обозначение документа)

МДК.02.01 Технология разработки программного обеспечения

Студент	ИСПП-21		08.12.2025	В.А. Екимов
	(Группа)	(Подпись)	(Дата)	(И.О. Фамилия)
Преподаватель			09.12.2025	Ю.С. Маломан
		(Подпись)	(Дата)	(И.О. Фамилия)

Архангельск 2025

СОДЕРЖАНИЕ

Перечень сокращений и обозначений 3
Введение 4
1 Сбор и анализ требований 6
1.1 Назначение и область применения 6
1.2 Постановка задачи 6
1.3 Выбор состава программных средств 8
2 Проектирование программного обеспечения 10
2.1 Проектирование интерфейса пользователя 10
2.2 Разработка архитектуры программного обеспечения 11
2.3 Проектирование базы данных 12
3 Разработка и интеграция модулей программного обеспечения 13
3.1 Разработка программных модулей 13
3.2 Реализация интерфейса пользователя 16
3.3 Разграничение прав доступа пользователей 18
3.4 Экспорт и импорт данных 20
4 Тестирование и отладка программного обеспечения 22
4.1 Структурное тестирование 22
4.2 Функциональное тестирование 23
5 Инструкция по эксплуатации программного обеспечения 24
5.1 Установка программного обеспечения 24
5.2 Инструкция по работе 25
Заключение 29
Список использованных источников 29

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем курсовом проекте применяются следующие сокращения и обозначения:

БД – база данных

ОС – операционная система

ПК – персональный компьютер

ПО – программное обеспечение

СУБД – система управления базами данных

УНФО – учет номерного фонда общежития

ЯП – язык программирования

API – интерфейс прикладного программирования

CSS – каскадная таблица стилей

DTO – Объект передачи данных

EFCore – фреймворк для доступа к данным

ERD – диаграмма взаимосвязей сущностей

HTML – язык гипертекстовой разметки

HTTP – протокол передачи гипертекста

HTTPS – безопасный протокол передачи гипертекста

ID - идентификатор

IDE – интегрированная среда разработки

REST – передача репрезентативного состояния

SDK – комплект для разработки программного обеспечения

ВВЕДЕНИЕ

В современном мире цифровые технологии стали основой для оптимизации и автоматизации процессов в самых разных сферах, включая управление жилым фондом и социальной инфраструктурой. Веб-приложения и информационные системы позволяют централизованно хранить данные, оперативно обрабатывать запросы и предоставлять актуальную информацию ответственным лицам в удобной форме. Это особенно важно для учреждений с регулярным документооборотом и высокой динамикой данных, таких как колледжи и их общежития, где точность и скорость доступа к информации напрямую влияют на качество административной работы.

Для учебных заведений среднего профессионального образования студенческие общежития являются критически важным элементом инфраструктуры. Однако во многих из них до сих пор используются устаревшие бумажные методы учёта: ручное ведение журналов, составление списков, проживающих на бумажных носителях, что приводит к значительным временным затратам персонала, высокому риску ошибок и низкой оперативности в критических ситуациях, таких как необходимость срочно определить состав людей в здании при эвакуации. Цифровизация этого процесса является ключевым шагом для решения указанных проблем, так как позволяет не только повысить эффективность работы администрации, но и кардинально улучшить контроль над использованием жилого фонда и обеспечить безопасность студентов.

Таким образом, разработка подсистемы учета номерного фонда общежития позволит автоматизировать рутинные операции, минимизировать человеческий фактор, обеспечить мгновенный доступ к актуальным данным и повысить общий уровень управляемости объектом.

Целью курсового проектирования является разработка программного решения для комендантов и ночных воспитателей, предназначенного для

автоматизации учета проживающих и оперативного управления инфраструктурой студенческого общежития.

Для успешной реализации проекта необходимо детально проработать каждый этап, уделяя особое внимание согласованности между компонентами. Анализ предметной области и аудитории станет фундаментом для проектирования гибкой и масштабируемой архитектуры. Последовательная разработка и всестороннее тестирование обеспечат создание надежной и удобной системы для всех пользователей. Для достижения поставленной цели требуется решить следующие задачи:

1) провести анализ предметной области и выявить функциональные требования к системе;
2) провести анализ целевой аудитории;
3) спроектировать архитектуру системы, включающую серверную часть, базу данных и клиентское веб-приложение;
4) реализовать БД для централизованного хранения данных;
5) реализовать серверную часть логики работы приложения для обработки информации;
6) разработать веб-приложение для коменданта и воспитателя;
7) провести тестирование функциональности.

1 Сбор и анализ требований

1.1 Назначение и область применения

Подсистема учета номерного фонда общежития предназначена для упрощения процесса расселения студентов, ведения учета данных о проживающих студентах и жилых помещениях. Её внедрение направлено на замену бумажного документооборота цифровым решением, что снизит время, затрачиваемое на формирование списков расселения, исключит ошибки при заселении и обеспечит быстрый доступ к информации о проживающих в общежитии.

Подсистема предназначена для использования в студенческих общежитиях учебных заведений среднего профессионального образования, например, в АКТ (ф) СПбГУТ. Основными пользователями являются комендант общежития и ночные воспитатели.

1.2 Постановка задачи

Необходимо разработать веб-приложение, которое предоставит доступ к следующей функциональности:

- авторизация пользователей;
- редактирование данных пользователей системы;
- редактирование данных о студентах и комнатах;
- добавление заметок о студентах;
- фильтрация и поиск данных о студентах и комнатах;
- формирование списка расселения студентов;
- формирование отчётов о проживающих студентах в формате .xlsx с возможностью применения фильтров.

При запуске веб-приложения отображается страница авторизации. Пользователь может авторизоваться в системе, если у него имеются данные

учетной записи, которые предоставляет организация. При успешной авторизации пользователь попадает на главную страницу приложения с доступной функциональностью в зависимости от роли пользователя в подсистеме.

Главная страница включает в себя навигационное бургер-меню, при помощи которого можно переключаться между функциональными возможностями приложения. На рисунке 1 изображена диаграмма вариантов использования подсистемы.

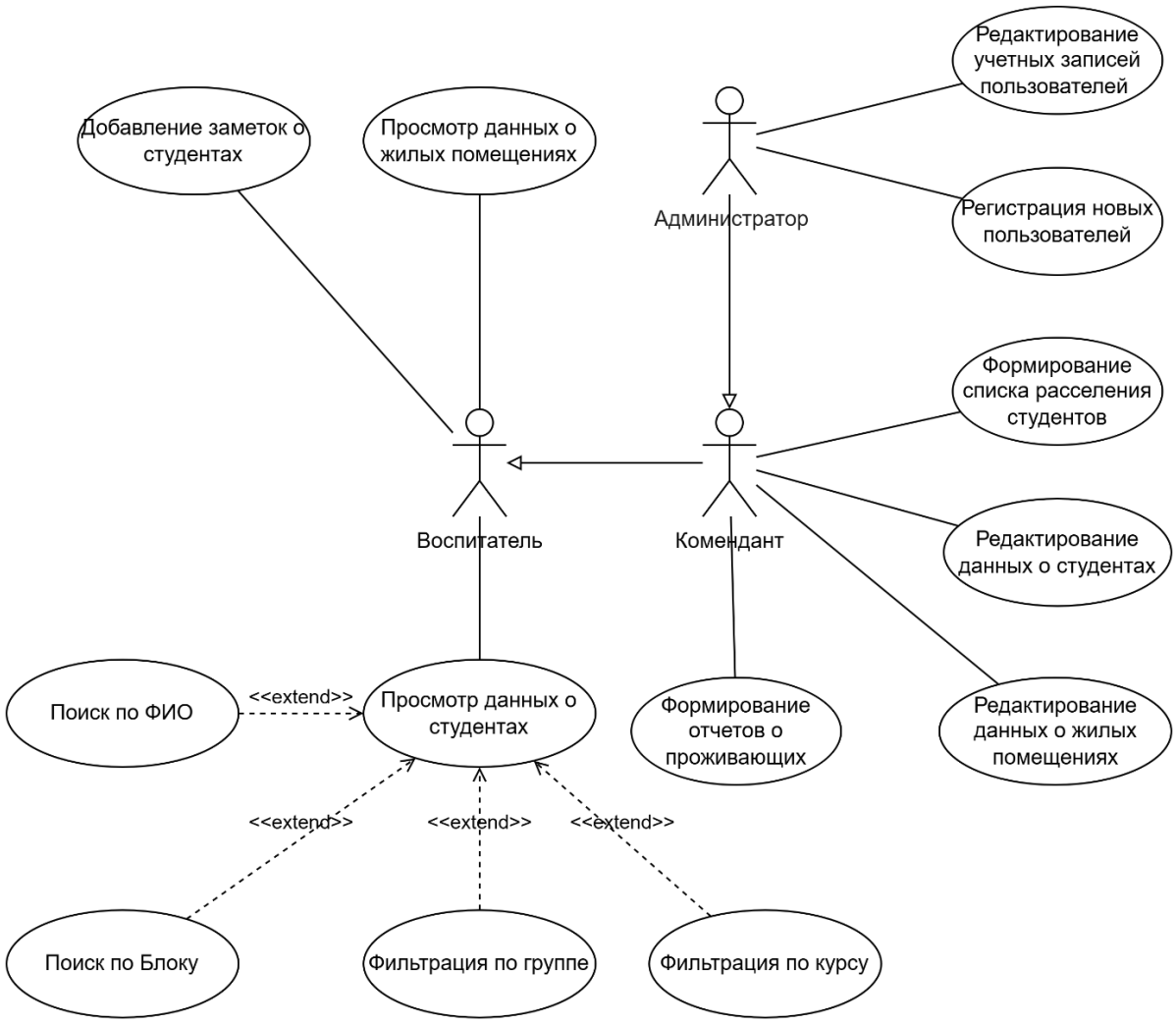

Рисунок 1 – Диаграмма вариантов использования подсистемы

Воспитатель имеет доступ к функциональности, позволяющей просматривать списки расселения студентов и информацию о проживающих в общежитии студентах и оставлять заметки о студентах.

Комендант имеет доступ к функциональности воспитателя, а также имеет права на редактирование данных о жилых помещениях общежития и студентах, формирование списка расселения студентов, формирование отчётов,

Администратор имеет доступ к функциональности коменданта и обладает расширенными правами доступа: может регистрировать новых пользователей, удалять и изменять записи о существующих, просматривать список всех пользователей.

1.3 Выбор состава программных средств

Согласно поставленной цели проекта требуется разработать веб-приложение для учета номерного фонда общежития.

Работа с веб-приложением будет осуществляться на устройствах с подключением по локальной сети. Для работы требуется браузер Google Chrome (версии 90 и выше), Mozilla Firefox (версии 88 и выше) или аналоги, которые обеспечивают полную поддержку стандартов HTML5 и CSS3, необходимых для корректной работы клиентской части приложения.

В качестве СУБД выбрана MySQL 8.0, являющаяся кроссплатформенным, стабильным и хорошо документированным решением, что делает ее подходящей для хранения структурированных данных.

Веб-приложение будет реализовано как одностраничное приложение, разрабатываемое с использованием фреймворка React и ЯП TypeScript. Этот выбор обусловлен возможностью реализации компонентной архитектуры интерфейса, обеспечением типобезопасности для повышения надежности кода и создания высокоинтерактивного пользовательского интерфейса.

Серверная логика реализуется на ЯП C# с использованием фреймворка ASP.NET Core. Этот стек технологий выбран за его надежность, высокую производительности, наличие встроенной поддержки для построения REST API, а также удобство интеграции с реляционными СУБД.

Для разработки серверной части подсистемы будет использоваться IDE Visual Studio 2022, так как она обеспечивает удобные инструменты для написания кода, отладки и тестирования.

Для разработки клиентской части подсистемы будет использоваться IDE Visual Studio Code, так как она поддерживает множество расширений для работы.

Для функционирования системы на стороне сервера достаточны следующие программные и технические средства:

- ОС Ubuntu Server версии 20.04 или аналоги;
- сервер БД: MySQL версии не ниже 8.0;
- процессор с частотой 2 ГГц;
- свободная оперативная память объемом 4 ГБ;
- ПО для работы API: .Net SDK версии не ниже 8.0.

Для функционирования системы на стороне клиента достаточны следующие программные и технические средства:

- веб-браузер: Google Chrome (версии 90 и выше), Mozilla Firefox (версии 88 и выше);
- процессор с частотой 2 ГГц;
- оперативная память: минимум 1 ГБ;
- постоянное интернет-подключение.

Приведенный набор программных средств обеспечивает эффективное функционирование веб-приложения для автоматизированного учета номерного фонда студенческого общежития.

2 Проектирование программного обеспечения

2.1 Проектирование интерфейса пользователя

В рамках разработки веб-приложения при помощи инструмента draw.io спроектирован интерфейс пользователя в виде набора wireframe, который позволяет представить его основные элементы и функциональность. Wireframe страниц «Авторизация», «Структура общежития», «Карточка студента» представлен на рисунке 2.

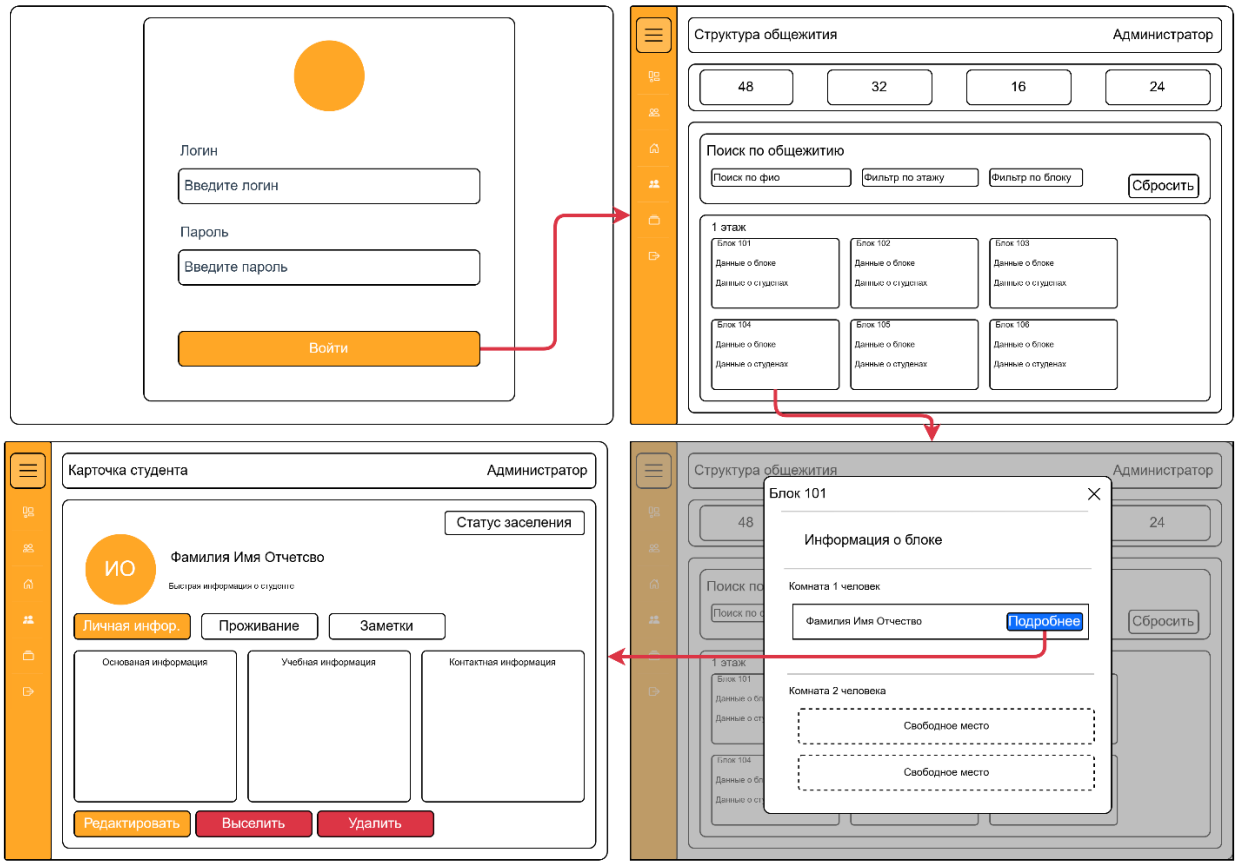

Рисунок 2 – draw.io. Wireframe основных страниц

Цветовая палитра веб-приложения должна соответствовать корпоративному стилю СПбГУТ, чьим филиалом является АКТ (ф) СПбГУТ.

Для веб-приложения выбраны следующие цвета:

- основной цвет: #FFA726;
- цвет фона: #FFFFFF;
- вторичный цвет фона: #F8F9FA;
- основной цвет текста: #2C3E50;
- вторичный цвет текста: #333333.

2.2 Разработка архитектуры программного обеспечения

Архитектура подсистемы построена на основе клиент-серверной модели и включает в себя следующие компоненты: веб-сайт, БД, API для взаимодействия с БД [5].

Диаграмма развертывания компонентов изображена на рисунке 4.

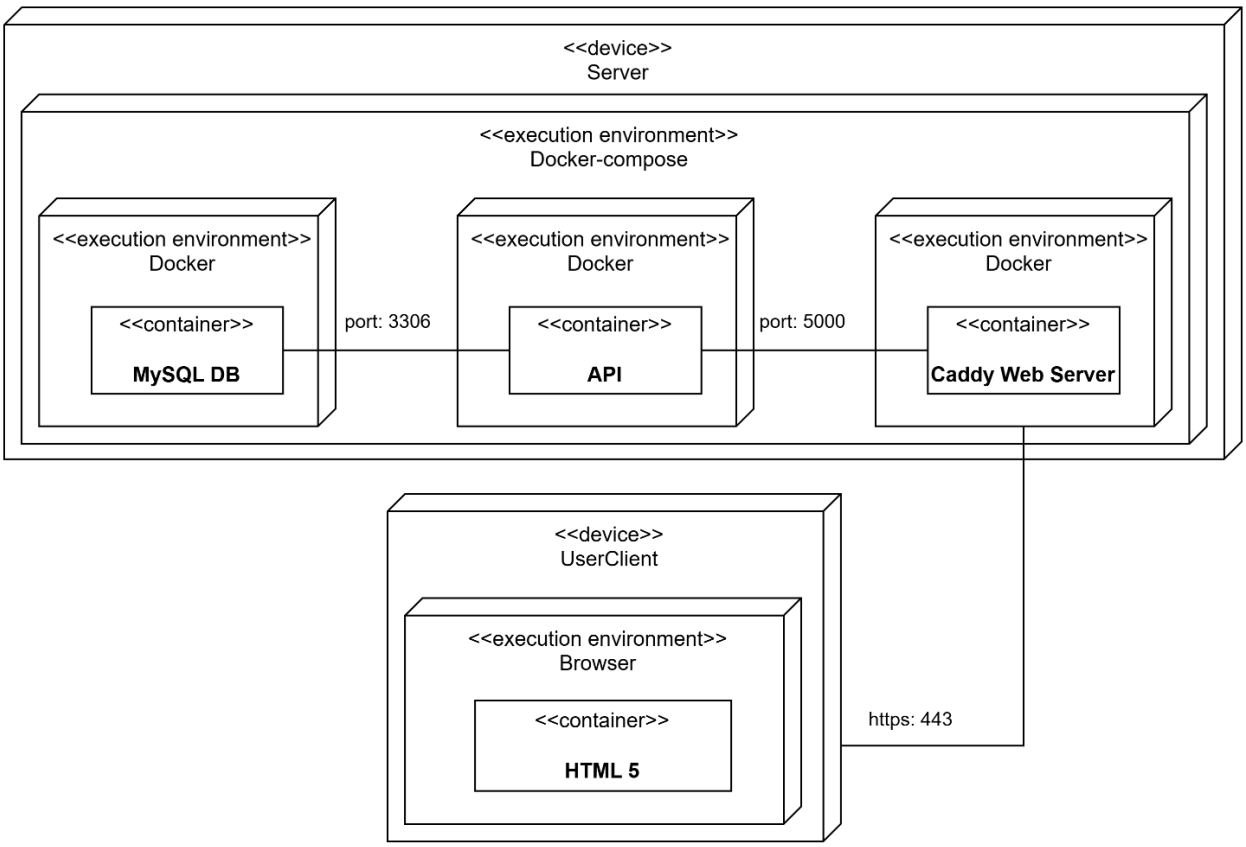

Рисунок 4 – Диаграмма развертывания компонентов

2.3 Проектирование базы данных

В рамках разработки веб-приложения при помощи MySQL Workbench спроектирована физическая модель БД для подсистемы учета номерного фонда общежития, представленная на рисунке 3 в виде ERD [3].

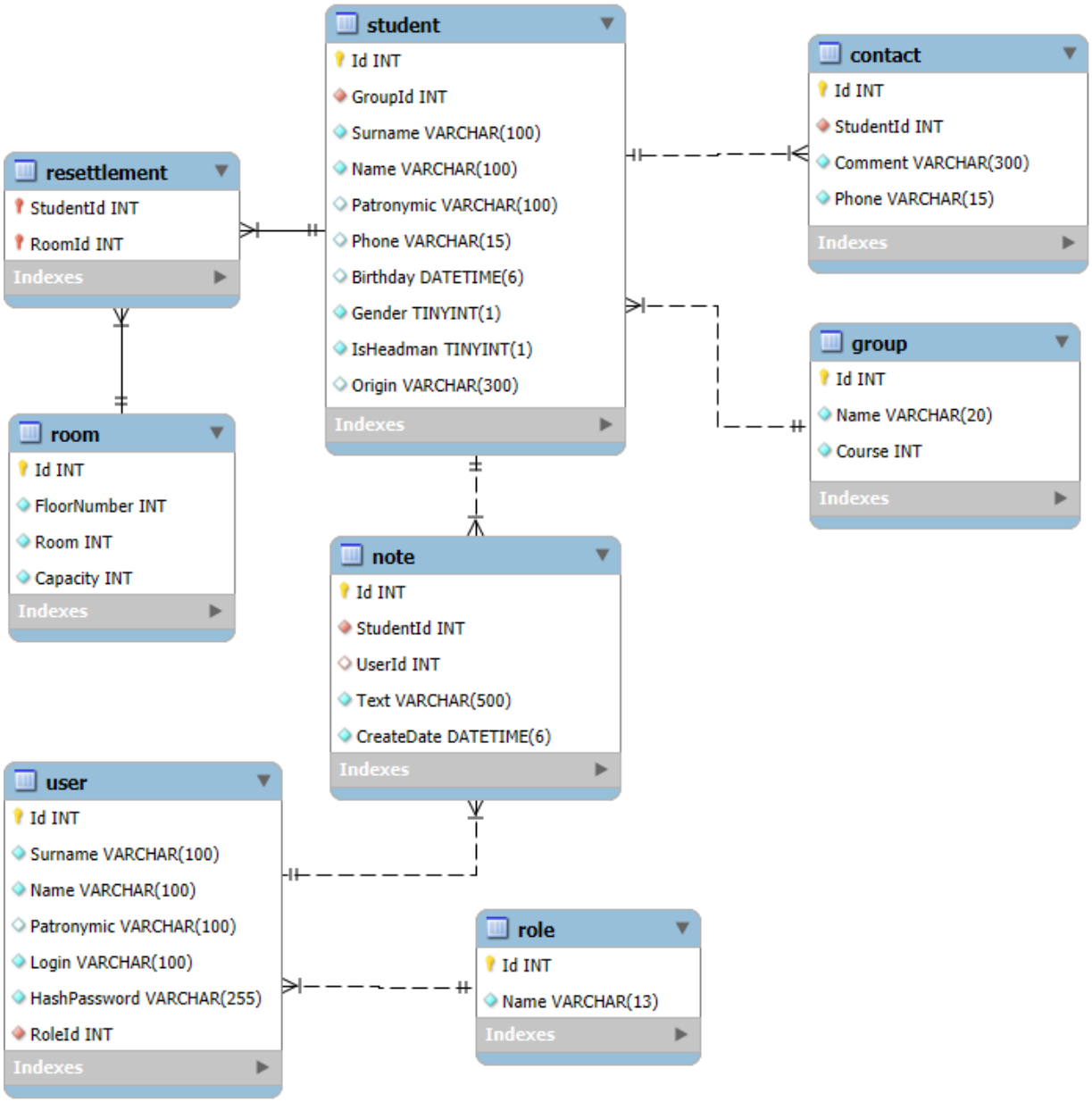

Рисунок 3 – MySQL Workbench. Физическая модель БД

3 Разработка и интеграция модулей программного обеспечения

3.1 Разработка программных модулей

В ходе курсового проектирования разработаны следующие приложения: веб-приложение на React с использованием фреймворка Vite в Visual Studio Code и Web-API приложение на C# в Visual Studio 2022 [2].

Обращение к информации, хранящейся в БД, осуществляется посредством Web-API приложения, с использованием библиотеки EFCore, созданными моделями таблиц БД, а также DTO объектами. Код метода для получения списка студентов представлен листингом 1.

Листинг 1 – Код метода для отправки GET-запроса на сервер

```
// Возвращает список всех студентов с информацией
// об их группе и комнатах.
// Сущности преобразуются в DTO с помощью метода ToDto()
[HttpGet]
[SwaggerOperation(
    Summary = "Получение списка всех студентов",
    Description = "Возвращает полный список студентов, включая
данные об их учебной группе и закреплённых комнатах.")]
[SwaggerResponse(
     StatusCodes.Status200OK,
     "Список студентов успешно получен.",
     Type = typeof(IEnumerable<StudentDto>))]
public async Task<ActionResult<IEnumerable<StudentDto>>>
GetAllStudents()
{
    var students = await _context.Students
        .Include(s => s.Group)    // Загружаем связанную учебную
группу
        .Include(s => s.Rooms)    // Загружаем список комнат, в
которых состоит студент (например, при заселении)
        .ToListAsync();

    return Ok(students.Select(s => s.ToDto())); // Преобразуем
сущности в DTO перед отправкой клиенту
}
```

Взаимодействие веб-приложения с сервером происходит при помощи HTTP-запросов к API, ответы возвращаются в формате JSON, в связи с чем разработана функция для выполнения HTTP-запросов к API, представленная листингом 2.

Листинг 2 – Код функции для выполнения HTTP-запросов к API

```
/**
 * Универсальная функция для выполнения HTTP-запросов к API.
 * Автоматически добавляет базовый URL и обрабатывает типичные
сценарии
 *
 * @template T — ожидаемый тип ответа.
 * @param url — путь относительно API_BASE_URL.
 * @param options — параметры запроса (метод, тело, заголовки и
т.д.).
 * @returns Промис с данными типа T или выбрасывает ошибку.
 */
async function request<T>(url: string, options: RequestInit =
{}): Promise<T> {
  const fullUrl = `${API_BASE_URL}${url}`;

  try {
    const response = await fetch(fullUrl, {
      headers: {
        'Content-Type': 'application/json',
        ...options.headers,
      },
      ...options,
    });

    // Обработка недействительного или просроченного токена
    if (response.status === 401) {
      handleUnauthorized(); // Очищает сессию и перенаправляет
на логин
      throw new Error('Токен недействителен. Вы будете
перенаправлены на страницу входа.');
    }

    // Успешный ответ (2xx)
    if (response.ok) {
      // Для ответов без тела (например, 204 No Content)
      if (response.status === 204) {
        return undefined as unknown as T;
      }

      // Получаем текстовое тело ответа
      const text = await response.text();
```

```
      // Пустой ответ — считаем допустимым, возвращаем undefined
      if (!text) {
        return undefined as unknown as T;
      }

      // Пытаемся распарсить JSON
      try {
        const data = JSON.parse(text);
        return data;
      } catch (parseError) {
        console.error("Ошибка парсинга JSON:", parseError);
        console.error("Полученный текст:", text);
        throw new Error(`Сервер вернул некорректные данные:
${parseError}`);
      }
    }

    // Обработка ошибок от сервера (4xx, 5xx)
    const errorData: ApiErrorDto = await
response.json().catch(() => ({
      timestamp: Date.now(),
      message: `Ошибка ${response.status}`,
      errorCode: response.status,
    }));

    const errorMessage = errorData.message || `Ошибка
${response.status}`;
    throw new Error(errorMessage);
  } catch (error: any) {
    // Сетевая ошибка (например, нет подключения к интернету или
сервер не отвечает)
    if (error.name === 'TypeError' &&
error.message.includes('fetch')) {
      throw new Error('Сервер недоступен. Попробуйте позже.');
    }
    // Пробрасываем все остальные ошибки выше
    throw error;
  }
}
```

Для получения дополнительных контактов студента реализован метод Web-API, который представлен листингом 3.

Листинг 3 – Код получения списка контактной информации о студенте

```
// Возвращает список контактной информации для студента по ID.
// Если студент не существует — возвращается ошибка 404.
```

```
// Контакты преобразуются в DTO для обеспечения чёткого и
безопасного API-контракта.
[HttpGet("{id}/contacts")]
[SwaggerOperation(
    Summary = "Получение контактной информации студента",
    Description = "Возвращает все контакты (телефон и
комментарий), привязанные к студенту по его идентификатору.")]
[SwaggerResponse(
     StatusCodes.Status200OK,
     "Контакты успешно получены.",
     Type = typeof(List<ContactDto>))]
[SwaggerResponse(StatusCodes.Status404NotFound, "Студент с
указанным ID не найден.", Type = typeof(ApiErrorDto))]
public async Task<ActionResult<List<ContactDto>>>
GetContacts(int id)
{
    // Проверяем существование студента по ID
    var studentExists = await _context.Students.AnyAsync(s =>
s.Id == id);
    if (!studentExists)
        return NotFound(new ApiErrorDto("Студент не найден",
StatusCodes.Status404NotFound));

    // Загружаем все контакты, связанные с этим студентом
    var contacts = await _context.Contacts
        .Where(c => c.StudentId == id)
        .ToListAsync();

    // Преобразуем сущности в DTO и возвращаем клиенту
    return Ok(contacts.Select(c => new ContactDto(c.Comment,
c.Phone)));
}
```

3.2 Реализация интерфейса пользователя

Интерфейс веб-приложения разработан с использованием одностраничной навигации, в приложении разработаны различные элементы управления, стили и виджеты для упрощения работы. Навигация в приложении организована через React Router, использующий древовидную структуру маршрутов, где каждый Route соответствует определенной странице. Для отображения общей информации в табличном виде разработан компонент CommonTable, который представлен рисунком 4. Код разработанного компонента CommonTable представлен листингом 4 [4].

№	ФИО	Группа	Курс	Пол	Блок	Телефон	День рождения	Проживание
1	Абрамова Софья Максимовна	ИСС-41	2	Ж	701	89991234590	20.06.2004	701 (2)
2	Андреев Роман	ССА-22	4	М	701	—	21.08.2002	701 (3)
3	Баранов Антон Викторович	ОИБ-51	1	М	701	89991234603	27.03.2005	701 (3)
4	Белов Кирилл Александрович	ИСС-31	3	М	701	89991234587	15.08.2003	701 (3)
5	Борисова Ангелина Романовна	РМТ-51	1	Ж	701	89991234610	18.05.2005	701 (2)

Рисунок 4 – УНФО. Вид компонента CommonTable

Листинг 4 – Код верстки компонента CommonTable

```
// Универсальный компонент таблицы
const CommonTable = <T extends Record<string, any>>({
    title, data, columns, emptyMessage = 'Данные не найдены',
className = '', enableSorting = false, onSortRequest,
sortConfig, rowAction, onRowClick, rowActionOpenOnRowClick =
false,
}: CommonTableProps<T>) => {

// Флаг, нужно ли рисовать столбец действий
    const hasRowAction = Boolean(rowAction);

return (
<div className={`${styles.tableWrapper} ${className}`}>
     ...
    <div className={styles.tableResponsive}>
        <table className={styles.table}>
            <thead>
                {/*Шапка таблицы*/}
                <tr>
                    {columns.map((column, index) => (
                        <th key={index}
className={column.className}>{renderColumnTitle(column)}</th>
                    ))}
                    {hasActions && <th>Действия</th>}
                </tr>
            </thead>
            <tbody>
                {/*Данные таблицы*/}
                {data.length ? (
                     data.map((item, rowIndex) => (
                        <tr key={rowIndex}>
                            {columns.map((column, colIndex) => (
```

```
                                <td key={colIndex}
className={column.className}>
                                    {column.render ?
column.render(item) : getValueByPath(item, column.key) ?? 'Нет'}
                                </td>
                            ))}
                            {hasActions && (
                                <td>
                                    <div
className={styles.actionButtons}>
                                        {actions!.map((action,
actionIndex) => (
                                            <React.Fragment
key={actionIndex}>{action.render(item)}</React.Fragment>
                                        ))}
                                    </div>
                                </td>
                            )}
                        </tr>
                    ))
                ) : (
                    <tr>
                        {/*Plaseholder при отсутсвии данных*/}
                        <td colSpan={columns.length +
(hasActions ? 1 : 0)} className="text-
center">{emptyMessage}</td>
                    </tr>
                )}
            </tbody>
        </table>
    </div>
</div>
);};

export default CommonTable;
```

3.3 Разграничение прав доступа пользователей

Разграничение прав доступа реализовано разделением пользователей на три роли:

- администратор;
- комендант;
- воспитатель.

Права определенной роли у сотрудника появляются только при правильном идентификаторе роли в поле RoleId. Фрагмент кода для разграничения прав доступа по маршрутам представлен листингом 5.

Листинг 5 – Фрагмент кода для разграничения прав доступа по маршрутам

```
// Основной массив маршрутов приложения
export const routes = [
  {
    // Корневой маршрут - страница входа
    path: "/", element: <LoginPage />,
  },
  {
    // Маршрут панели управления с защитой доступа
    path: "/dashboard",
    element: (
      <ProtectedRoute>
        <DashboardLayout />
      </ProtectedRoute>
    ),
    // Мета-данные для маршрута
    handle: { title: 'Панель управления' },
    // Вложенные маршруты внутри дашборда
    children: [
      // Раздел структуры общежития (требуется роль educator)
      { path: "structure", element: <></>, handle: { title:
'Структура общежития', requiredRole: 'educator' } },
      { path: "students", element: <StudentsLayout />, handle: {
title: 'Студенты', requiredRole: 'educator' } },
      { path: "students/:studentId", element: <StudentCardLayout
/>, handle: { title: 'Карточка студента', requiredRole:
'educator' } },

      // Раздел студентов (требуется роль commandant)
      { path: "settlement", element: <></>, handle: { title:
'Расселение', requiredRole: 'commandant' } },

      // Раздел пользователей (требуется роль admin)
      { path: "users", element: <UsersLayout />, handle: {
title: 'Пользователи', requiredRole: 'admin' } },

      // Маршрут по умолчанию для /dashboard - редирект на
структуру
      { index: true, element: <Navigate to="structure" replace
/> },
    ],
  },
  // Маршрут для обработки несуществующих страниц (404)
```

```
  { path: "*", element: <NotFoundLayout /> },
];
```

3.4 Экспорт и импорт данных

Веб-приложение также предоставляет доступ к экспорту и импорту данных о студентах в формате *.xlsx. Код метода handleExportToExcel представлен листингом 6. Также в веб-приложении присутствует возможность скачать шаблон для импорта данных. Код метода для создания шаблона представлен листингом 7.

Листинг 6 – Код метода handleExportToExcel

```
const handleExportToExcel = () => {
    // Подготовка данных для экспорта из processedStudents
    const dataForExport = processedStudents.map(student => ({
        'ФИО': `${student.surname || ''} ${student.name || ''}
${student.patronymic || ''}`.trim() || 'Нет',
        'Группа': student.group?.name || 'Нет',
        'Курс': student.group?.course || 'Нет',
        'Пол': student.gender ? 'Мужчина' : 'Женщина',
        'Блок': student.blockNumber || 'Нет',
        'Телефон': student.phone || 'Нет',
        'День рождения': new Intl.DateTimeFormat('ru-RU', { day:
'numeric', month: 'long', year: 'numeric' }).format(new
Date(student.birthday)),
    }));

    // Создание книги Excel
    const wb = XLSX.utils.book_new();
    // Создание листа из данных
    const ws = XLSX.utils.json_to_sheet(dataForExport);
    // Добавление листа в книгу
    XLSX.utils.book_append_sheet(wb, ws, "Студенты");

    // Генерирование и скачивание файла
    XLSX.writeFile(wb, `Список_студентов_${new
Date().toISOString().slice(0, 10)}.xlsx`);
};
```

Листинг 7 – Код метода создания handleDownloadTemplate

```
const handleDownloadTemplate = useCallback(() => {
        //Создание тестовой записи на странице
        const templateSheet = XLSX.utils.aoa_to_sheet([
            IMPORT_EXPECTED_HEADERS,
            ['Иванов Иван Иванович', 'ИС-21', '3', 'М', 'г.
Пермь', '8XXXXXXXXXX', '12.08.2002'],
        ]);

        //Импорт страницы с доступными группами
        const groupsSheetData = [
            ['Группа', 'Курс'],
            ...groups
                .slice()
                .sort((a, b) => (a.name ||
'').localeCompare(b.name || '', 'ru'))
                .map(group => [group.name ?? '', group.course ??
'']),
        ];
        const groupsSheet =
XLSX.utils.aoa_to_sheet(groupsSheetData);

        const workbook = XLSX.utils.book_new();
        XLSX.utils.book_append_sheet(workbook, templateSheet,
'Шаблон');
        XLSX.utils.book_append_sheet(workbook, groupsSheet,
'Группы');
        XLSX.writeFile(workbook,
'Шаблон_импорта_студентов.xlsx');
    }, [groups]);
```

4 Тестирование и отладка программного обеспечения

4.1 Структурное тестирование

Во время курсового проектирования проведено тестирование методом белого ящика для методов Web-API, результаты представлены в таблице 1 [1].

Таблица 1 – Результаты тестирования конечной точки создание заметки

Действие	Ожидаемый результат	Фактический результат

1) Убедиться, что студент с ID 0 не существует
2) Выполнить POST: api/v1/Notes запрос в API с телом:
 	
{
"studentId": 0,
"userId": 0,
"text": "Тестовая заметка"
}
	Получение HTTP-ответа 404 Not Found. Тело ответа содержит время получения ошибки, информацию о коде ответа, а также информацию о том, что студент не найден.	Совпадает с ожидаемым

1) Убедиться, что студент с ID 1 существует
2) Выполнить POST: api/v1/Notes запрос в API с телом:
 	
{
"studentId": 1,
"userId": 0,
"text": "Тестовая заметка"
}
	Получение HTTP-ответа 404 Not Found. Тело ответа содержит время получения ошибки, информацию о коде ответа, а также информацию о том, что пользователь не найден.	Совпадает с ожидаемым

1) Убедиться, что студент и пользователь с ID 1 существуют
2) Выполнить POST: api/v1/Notes запрос в API с телом:
 	
{
"studentId": 1,
"userId": 1,
"text": "Тестовая заметка"
}
	Получение HTTP-ответа 201 Created. Тело ответа содержит ID заметки, ID студента, текст заметки, а также информацию об авторе и его роли.	Совпадает с ожидаемым

4.2 Функциональное тестирование

Во время курсового проектирования проведено функциональное тестирование веб-приложения методом черного ящика, результаты тестирования представлены в таблице 2.

Таблица 2 – Результаты тестирования приложения методом черного ящика

Действие	Ожидаемый результат	Фактический результат
Нажать на бургер-кнопку, нажать на вкладку «Студенты»	Переход на экран «Студенты»	Совпадает с ожидаемым
Нажать на бургер-кнопку, нажать на кнопку «Студенты», нажать на кнопку «Общежитие»	Переход на экран «Общежитие»	Совпадает с ожидаемым
Нажать на бургер-кнопку, нажать на кнопку «Расселение», нажать на кнопку «Общежитие»	Сброс выбранной вкладки на странице «Общежитие», переход ко вкладке «Структура»	Совпадает с ожидаемым
Нажать на кнопку «Расселение», обновить страницу	Сброс выбранной вкладки не происходит	Совпадает с ожидаемым
Нажать на кнопку «Расселение», нажать на строку записи о незаселённом студенте	Студент автоматически выбирается в поле «Студент»	Совпадает с ожидаемым
Нажать на карточку блока №101	Открытие модального окна информации о блоке №101	Совпадает с ожидаемым
Нажать на карточку блока, в котором есть хотя бы одно свободное место, нажать на область с надписью «Свободное место»	Переход на вкладку «Расселение», автоматическое заполнение полей «Этаж» и «Комната» значениями пустого места	Совпадает с ожидаемым
Нажать на кнопку «Расселение», нажать на строку записи о незаселённом студенте Екимове	В поле «Комната» отображаются пустые комнаты и комнаты, в которых проживают студенты того же пола	Совпадает с ожидаемым

По результатам тестирования можно сделать вывод, что разработанное ПО работает корректно и согласно ожиданиям.

5 Инструкция по эксплуатации программного обеспечения

5.1 Установка программного обеспечения

Для функционирования подсистемы на стороне сервера достаточны следующие программные и технические средства:

- ОС Ubuntu Server версии 20.04 или аналоги;
- процессор с частотой 2 ГГц;
- свободная оперативная память объемом 4 ГБ;
- дополнительные компоненты: Docker, docker-compose, MySQL 8.0 (или выше), Caddy (или Nginx), .Net SDK версии не ниже 8.0 для работы API.

Для установки серверной части необходимо выполнить следующую последовательность действий:

8) поместить файлы проекта на сервер;

9) добавить в корень проекта файл .env и задать значения переменных окружения (листинг 8);

10) в корневой папке проекта через терминал выполнить команду docker compose up -d.

Листинг 8 – Пример файла .env

```
DB_ROOT_PASSWORD=YOUR_ROOT_PASSWORD
DB_NAME=YOUR_DB_NAME //Имя БД
DB_USER=YOUR_DB_USER //Имя пользователя в БД
DB_PASSWORD=YOUR_PASSWORD //Пароль пользователя в БД
ASPNETCORE_ENVIRONMENT=Production
FRONTEND_API_URL=https://api.YOUR_DOMEN_NAME
CORS_ORIGINS=http://localhost:5173,https://YOUR_DOMEN_NAME
DB_VOLUME_NAME=YOUR_DB_VOLUME_NAME
```

Для функционирования веб-приложения на стороне клиента достаточны следующие программные и технические средства:

- веб-браузер: Google Chrome (версии 90 и выше), Mozilla Firefox (версии 88 и выше);
- процессор с частотой 2 ГГц;
- оперативная память: минимум 1 ГБ;
- постоянное интернет-подключение.

В веб-приложении используются следующие данные для авторизации:

- логин – admin;
- пароль – admin.

5.2 Инструкция по работе

После открытия веб-приложения пользователь попадает на страницу авторизации, где ему необходимо ввести учетные данные для входа в систему. Страница авторизации представлена на рисунке 4. После авторизации у пользователя определяется его роль в системе: администратор, комендант, воспитатель. В зависимости от роли пользователь имеет разные права на доступ к функциональности системы.

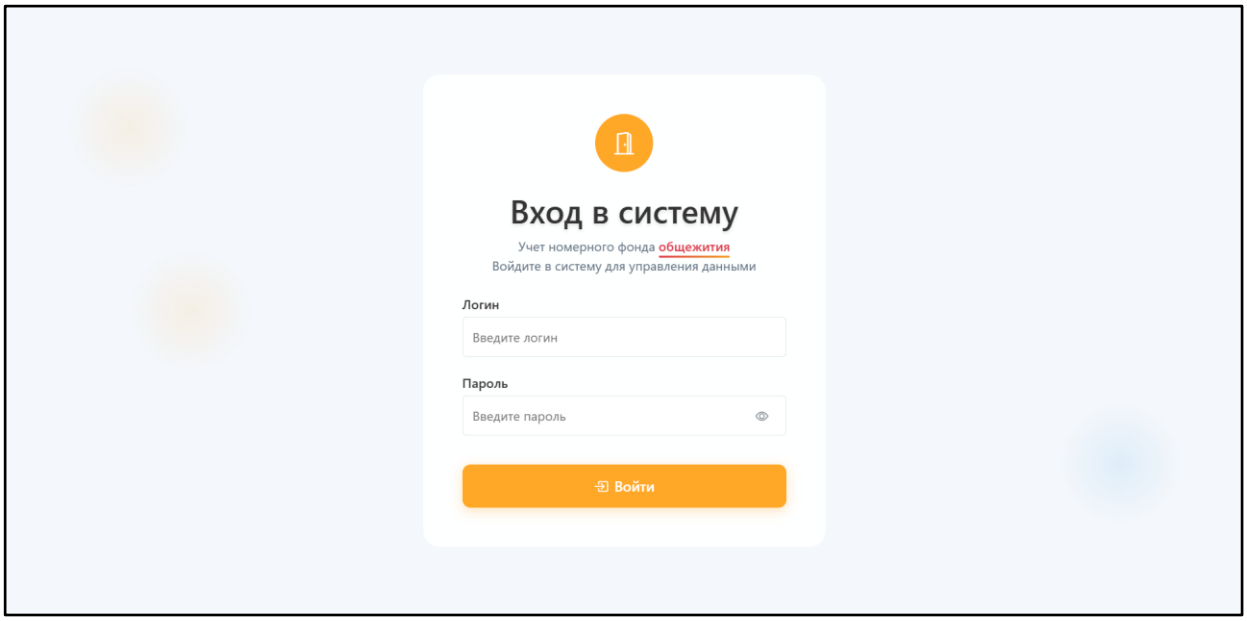

Рисунок 4 – УНФО. Вид страницы авторизации

После успешной авторизации пользователь попадает на страницу Структуры общежития (рисунок 5), на которой отображается общая статистика по студентами и жилым местам. Имеется возможность адаптивного поиска информации о студентах, этажах и блоках, а также отображается общая структура блоков общежития.

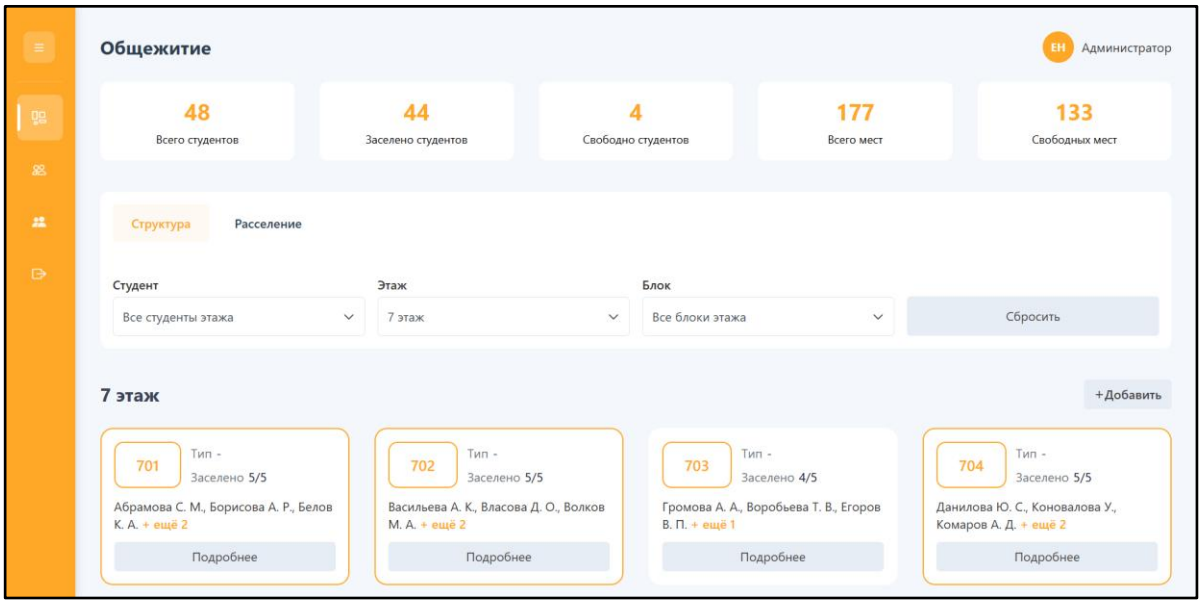

Рисунок 5 – УНФО. Вид страницы структура общежития

Для создания новой комнаты необходимо нажать на кнопку «+Добавить», при нажатии открывается небольшое модальное окно с тремя полями ввода «Номер этажа», «Номер комнаты», «Вместительность». При создании новой комнаты так же автоматически формируются блоки. Формат нумерации блока определяется номером этажа и номером комнаты. Формат нумерации комнат также определяется вместительностью. Для получения подробной информации о проживающих в блоке студентах пользователю необходимо нажать на карточку блока или на кнопку «Подробнее», после чего открывается модальное окно информации о блоке (рисунок 6).

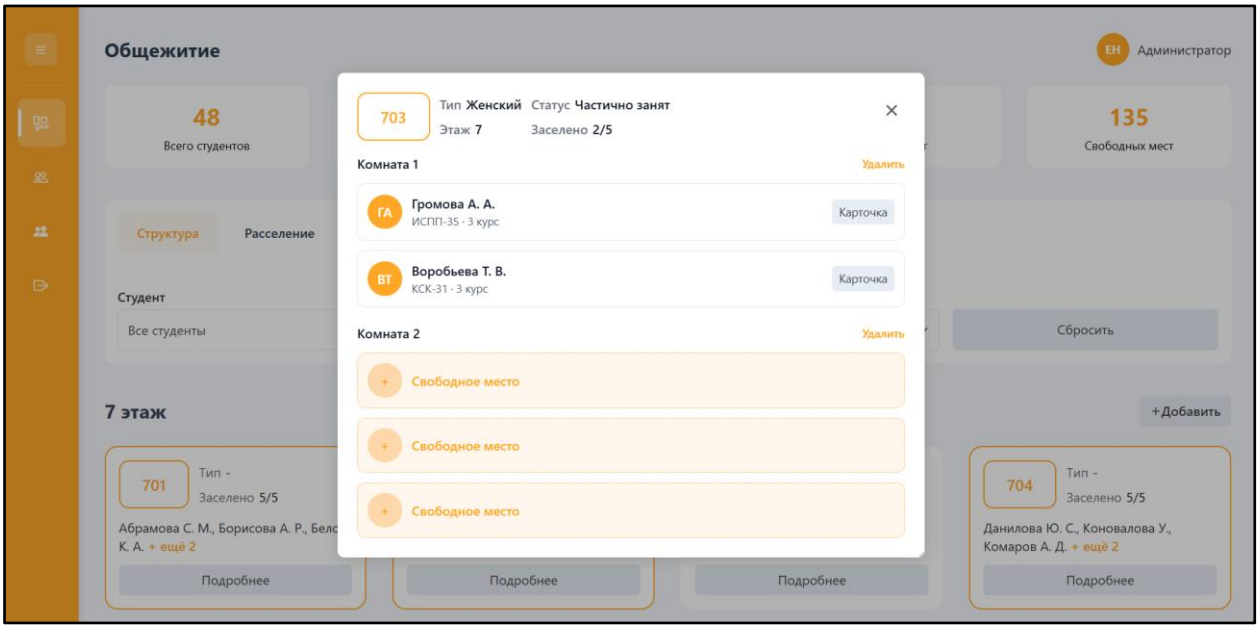

Рисунок 6 – УНФО. Вид модального окна блока

В модальном окне блока отображается общая статистика по блоку, заселенные студенты и свободные места. При нажатии на свободное место пользователь автоматически перенаправляется во вкладку «Расселение», где автоматически подставляются значения выбранного свободного места (рисунок 7).

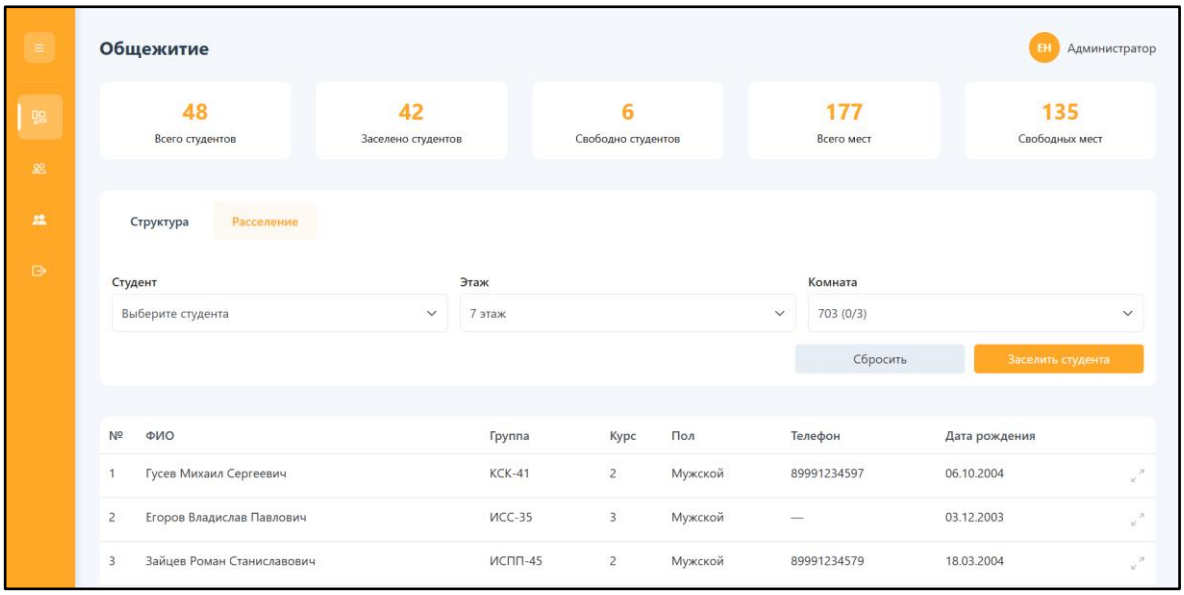

Рисунок 7 – УНФО. Вид страницы расселения общежития

На вкладке «Расселение» реализована функциональность быстрого выбора студентов посредством нажатия на строку таблицы, также по правилам расселения студентов студенты разных полов не могут быть заселены в одну комнату.

Также при нажатии на кнопку «Карточка», находящуюся в области информации о заселенном студенте, имеется возможность перехода к странице «Карточка студента», на которой представлена более подробная информация о студенте, имеется возможность оставлять заметки и просматривать информацию о соседях. Страница «Карточка студента» представлена на рисунке 8.

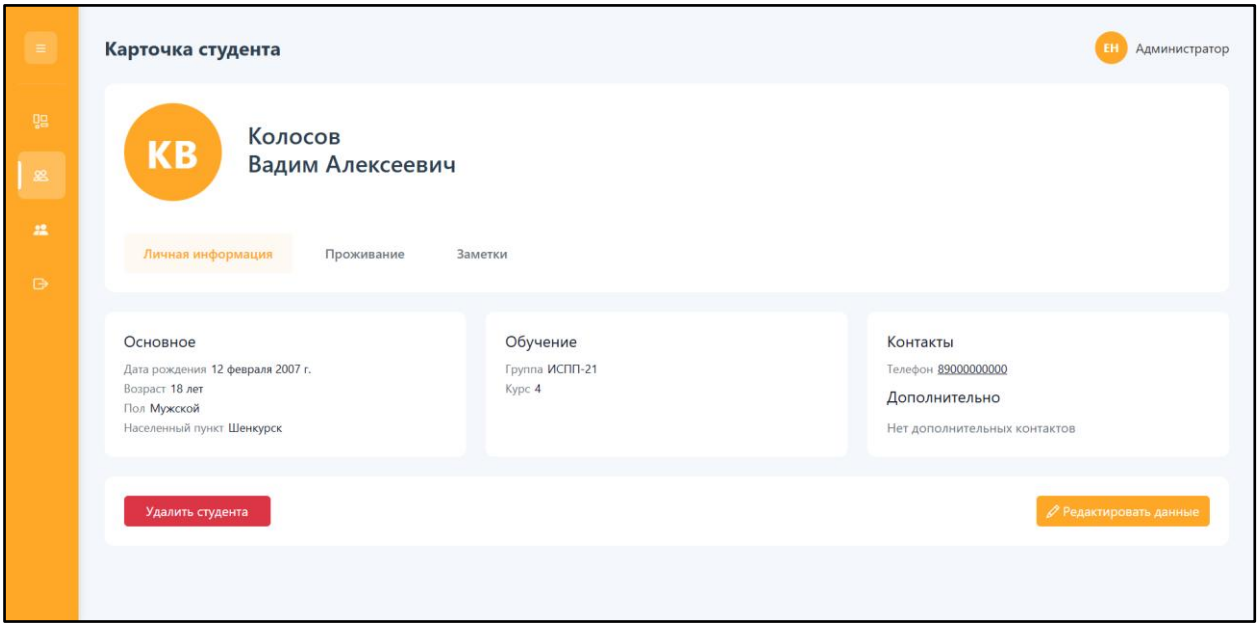

Рисунок 8 – УНФО. Вид страницы карточка студента

ЗАКЛЮЧЕНИЕ

В рамках курсового проектирования успешно разработана подсистема учета номерного фонда общежития. Созданное веб-приложение заменяет устаревшие ручные методы работы на централизованную цифровую систему, что напрямую решает такие задачи как сокращение временных затрат, минимизация ошибок и обеспечение оперативного доступа к информации.

Цель проекта разработка программного решения для автоматизации учета проживающих и оперативного управления инфраструктурой студенческого общежития достигнута.

В процессе выполнения решены все ключевые задачи:

1) проведен анализ предметной области и выявлены функциональные требования к системе;
2) проведен анализ целевой аудитории;
3) спроектирована архитектура системы, включающая серверную часть, БД и клиентское веб-приложение;
4) реализована БД для централизованного хранения данных;
5) реализована серверная часть логики работы приложения для обработки информации;
6) разработано веб-приложение для коменданта и воспитателя;
7) проведено тестирование функциональности.

Итогом проделанной работы является созданное программное решение. Разработанная подсистема представляет собой не просто теоретический прототип, а практически ориентированный инструмент, внедрение которого способно кардинально изменить текущие процессы. Выполнение проекта от анализа требований до тестирования позволило пройти полный жизненный цикл разработки ПО. Приобретенные практические навыки в проектировании серверной логики и веб-интерфейса, а также в их интеграции, составляют ценный опыт, подтверждающий готовность к реализации реальных проектов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бек, К. Экстремальное программирование: разработка через тестирование. – Санкт-Петербург : Питер, 2021. – 224 с. – URL: https://ibooks.ru/bookshelf/376974/reading (дата обращения: 01.11.2025). – Режим доступа: для зарегистрир. пользователей. – Текст : электронный.

2. Гагарина, Л. Г. Технология разработки программного обеспечения : учебное пособие / Л. Г. Гагарина, Е. В. Кокорева, Б. Д. Сидорова-Виснадул ; под ред. Л. Г. Гагариной. – Москва : ФОРУМ : ИНФРА-М, 2025. – 400 с. – URL: https://znanium.ru/catalog/product/2178802 (дата обращения: 12.11.2025). – Режим доступа: по подписке. – Текст : электронный.

3. Мартишин, С. А. Базы данных. Практическое применение СУБД SQL и NoSQL-типа для проектирования информационных систем : учебное пособие / С. А. Мартишин, В. Л. Симонов, М. В. Храпченко. – Москва : ФОРУМ : ИНФРА-М, 2024. – 368 с. – URL: https://znanium.ru/catalog/product/2096940 (дата обращения: 23.11.2025). – Режим доступа: по подписке. – Текст : электронный.

4. Тидвелл, Д. Разработка интерфейсов. Паттерны проектирования. 3-е изд. – Санкт-Петербург : Питер, 2022. – 560 с. – URL: https://ibooks.ru/bookshelf/386796/reading (дата обращения: 23.11.2025). – Режим доступа: для зарегистрир. пользователей. – Текст : электронный.

5. Федорова, Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности : учебное пособие. – Москва : КУРС : ИНФРА-М, 2024. – 336 с. – URL: https://znanium.ru/catalog/product/2083407 (дата обращения: 24.11.2025). – Режим доступа: по подписке. – Текст : электронный.